中国传统年画 Stable Diffusion LoRA 训练项目

本项目使用 kohya-ss/sd-scripts 训练 Stable Diffusion 1.5 LoRA 模型，用于生成中国传统年画风格的图像。

📋 目录

- 项目概述
- 环境配置
- 目录结构
- 快速开始
- 训练流程
- 推理使用
- 参数说明
- 训练结果
- 常见问题
- 文件说明

项目概述

训练目标

训练一个 LoRA 模型，使 Stable Diffusion 能够生成中国传统年画风格的图像。

数据集信息

- **图片数量**: 162 张
- **图片格式**: JPG
- **原始分辨率**: 主要为 135x180
- **训练分辨率**: 448x448 (使用 bucket 支持多分辨率)
- **标签**: 中文描述 + 英文触发词 "nianha"

训练配置

- **基础模型**: Stable Diffusion 1.5
- **LoRA Rank**: 24
- **LoRA Alpha**: 12
- **训练 Epochs**: 3
- **批次大小**: 6
- **学习率**: 1e-4 (UNet) / 5e-5 (Text Encoder)

环境配置

硬件要求

- **GPU**: NVIDIA L40S (46GB VRAM)
- **CUDA**: 12.1

- **系统**: Linux (Ubuntu)

软件环境

已配置的 Conda 环境

```
环境名称: sd-lora
Python: 3.10.19
PyTorch: 2.1.2 (CUDA 12.1)
```

主要依赖包

```
torch==2.1.2
torchvision==0.16.2
diffusers==0.25.0
transformers==4.44.0
accelerate==0.30.0
safetensors==0.4.2
xformers==0.0.23.post1
bitsandbytes
numpy<2
```

首次环境搭建

如果需要从头搭建环境：

```
# 1. 创建 conda 环境
conda create -n sd-lora python=3.10 -y
conda activate sd-lora

# 2. 安装 PyTorch (CUDA 12.1)
pip install torch==2.1.2 torchvision==0.16.2 --index-url
https://download.pytorch.org/whl/cu121

# 3. 克隆并安装 sd-scripts
cd /home/ubuntu
git clone https://github.com/kohya-ss/sd-scripts.git
cd sd-scripts
pip install -r requirements.txt
pip install 'numpy<2'  # 降级 numpy 避免兼容性问题

# 4. 配置 accelerate
accelerate config
# 选择: This machine / No distributed / No / No / No / all / fp16
```

目录结构

```
/home/ubuntu/lora-training/
├── dataset/                          # 数据集目录
│   └── 10_nianha/                   # DreamBooth 格式（10x重复）
│       ├── image1.jpg               # 训练图片
│       ├── image1.txt               # 对应标签（中文 + "nianha"）
│       ├── image2.jpg
│       ├── image2.txt
│       └── ...                      # 共 162 对图片和标签
│
├── models/                           # 基础模型目录
│   └── sd-v1-5.safetensors         # SD 1.5 基础模型（7.2GB）
│
├── output/                           # 训练输出目录
│   ├── nianha_lora.safetensors     # 最终 LoRA 模型
│   ├── nianha_lora-step00000273.safetensors  # Epoch 1 检查点
│   ├── nianha_lora-step00000546.safetensors  # Epoch 2 检查点
│   └── nianha_lora-step00000819.safetensors  # Epoch 3 检查点
│
├── logs/                             # 训练日志目录
│   └── nianha/                     # TensorBoard 日志
│
├── generated_images/                 # 推理生成图片目录
│
├── train_lora_nianha.sh            # 训练脚本（主要）
├── add_trigger_word.py             # 数据预处理：添加触发词
├── inference_demo.py               # 推理脚本（diffusers 版本）
├── inference_kohya.py              # 推理脚本（kohya 版本）
└── README.md                         # 本文档
```

快速开始

1. 数据准备（已完成）

数据集已经准备好，位于 `dataset/10_nianha/` 目录。

如果需要准备新数据集：

```
# 1. 解压数据到 dataset/train/
unzip upload.zip -d dataset/train/

# 2. 添加英文触发词到所有标签
python add_trigger_word.py
# 这会将 "nianha" 添加到所有 .txt 文件开头

# 3. 重组为 DreamBooth 格式
mv dataset/train/* dataset/10_nianha/
# 目录名 "10_" 表示每张图片重复训练 10 次
```

2. 下载基础模型（已完成）

SD 1.5 模型已下载到 `models/sd-v1-5.safetensors`。

如需重新下载：

```
cd /home/ubuntu/lora-training/models
wget https://huggingface.co/runwayml/stable-diffusion-v1-5/resolve/main/v1-5-pruned.safetensors \
     -O sd-v1-5.safetensors
```

3. 开始训练

```
# 激活环境
source /home/ubuntu/miniconda3/bin/activate sd-lora

# 进入项目目录
cd /home/ubuntu/lora-training

# 运行训练脚本
bash train_lora_nianha.sh
```

训练将会：

- 显示训练进度和 loss
- 每个 epoch 结束保存检查点（步数 273, 546, 819）
- 在第 820 步保存最终模型

4. 推理测试

训练完成后，使用以下命令生成图片：

```
# 激活环境
source /home/ubuntu/miniconda3/bin/activate sd-lora

# 运行推理脚本
cd /home/ubuntu/lora-training
python inference_demo.py
```

生成的图片会保存到 `generated_images/` 目录。

训练流程

完整训练流程

1. **数据准备**
    - 收集 162 张年画图片
    - 为每张图片编写中文标签
    - 添加触发词 "nianha"
    - 重组为 DreamBooth 目录结构
2. **环境配置**
    - 创建 conda 环境
    - 安装依赖包
    - 配置 accelerate
3. **模型下载**
    - 下载 SD 1.5 基础模型
4. **训练参数调整**
    - 编辑 `train_lora_nianha.sh`
    - 根据数据集调整参数
5. **执行训练**
    - 运行训练脚本
    - 监控 loss 变化
    - 等待训练完成（约 50 分钟）
6. **模型测试**
    - 使用推理脚本生成图片
    - 评估模型效果
    - 调整 LoRA 权重优化效果

推理使用

方式 1: 使用 inference_demo.py（推荐）

```
source /home/ubuntu/miniconda3/bin/activate sd-lora
cd /home/ubuntu/lora-training
python inference_demo.py
```

脚本特点:

- 使用 diffusers 库，现代化 API
- 支持 xformers 内存优化
- 自动加载 LoRA 权重
- 生成 4 张示例图片

方式 2: 使用 inference_kohya.py

```
source /home/ubuntu/miniconda3/bin/activate sd-lora
cd /home/ubuntu/lora-training
python inference_kohya.py
```

脚本特点:

- 使用 kohya-ss 原生推理工具
- 与训练工具完全兼容
- 调用 `gen_img_diffusers.py`

方式 3: 使用 WebUI/ComfyUI

1. 复制 LoRA 模型到对应目录:

   ```
   # Stable Diffusion WebUI
   cp output/nianha_lora.safetensors \
      /path/to/stable-diffusion-webui/models/Lora/

   # ComfyUI
   cp output/nianha_lora.safetensors \
      /path/to/ComfyUI/models/loras/
   ```

2. 在界面中:

   - 选择 SD 1.5 基础模型
   - 加载 `nianha_lora.safetensors`
   - 设置 LoRA 权重: 0.6-0.9
   - 在提示词中包含 `nianha`

推理参数配置

编辑推理脚本顶部的参数:

```
# LoRA 参数
LORA_WEIGHT = 0.8           # LoRA 权重 (0.6-0.9)

# 推理参数
NUM_INFERENCE_STEPS = 30    # 推理步数 (20-50)
GUIDANCE_SCALE = 7.5        # CFG 引导强度 (5-10)
IMAGE_WIDTH = 512           # 图片宽度
IMAGE_HEIGHT = 512          # 图片高度
SEED = 42                   # 随机种子 (None 为随机)

# 提示词
PROMPTS = [
```

```
    "nianha, 传统年画, 门神形象, 威武雄壮, 红色调",
    "nianha, 年画风格, 仕女图, 古典美女, 细腻笔触",
    # ... 添加更多提示词
]

NEGATIVE_PROMPT = "low quality, blurry, distorted, modern, ..."
```

提示词编写技巧

必须包含触发词 `nianha`!

示例:

```
nianha, 传统年画, 门神形象, 威武雄壮, 红色调, 细腻笔触, 高质量
nianha, 年画艺术, 福娃抱鱼, 喜庆祥和, 春节氛围
nianha, 中国风, 仕女图, 古典美女, 宫廷服饰
nianha, 戏曲人物, 京剧脸谱, 色彩鲜艳, 传统文化
```

Negative 提示词:

```
low quality, blurry, distorted, modern, photograph, realistic,
3d render, western style, anime, cartoon
```

参数说明

训练脚本参数 (train_lora_nianha.sh)

路径配置

```
PRETRAINED_MODEL="/home/ubuntu/lora-training/models/sd-v1-5.safetensors"
TRAIN_DATA_DIR="/home/ubuntu/lora-training/dataset"  # 包含 10_nianha 的父目录
OUTPUT_DIR="/home/ubuntu/lora-training/output"
LOGGING_DIR="/home/ubuntu/lora-training/logs"
OUTPUT_NAME="nianha_lora"
```

LoRA 参数

```
NETWORK_DIM=24              # LoRA rank (艺术风格推荐 16-32)
NETWORK_ALPHA=12            # LoRA alpha (通常是 rank 的一半)
```

调整建议:

- **风格学习**: rank 16-32
- **角色学习**: rank 32-64
- **概念学习**: rank 8-16
- **精细细节**: 增大 rank
- **泛化能力**: 减小 rank

训练参数

```
RESOLUTION=448              # 训练分辨率
BATCH_SIZE=6                # 批次大小
MAX_TRAIN_STEPS=820         # 总训练步数 (3 epochs)
LEARNING_RATE=1e-4          # UNet 学习率
TEXT_ENCODER_LR=5e-5        # Text Encoder 学习率
```

调整建议:

- **分辨率**:
  - 小图源 → 448 或 512
  - 高清图源 → 512 或 768
- **批次大小**:
  - 46GB VRAM → 4-8
  - 24GB VRAM → 2-4
  - 显存不足时减小
- **训练步数**:
  - 计算公式: `图片数 × 重复次数 ÷ 批次大小 × epochs`
  - 本项目: `162 × 10 ÷ 6 × 3 = 810`
- **学习率**:
  - 过拟合 → 降低到 5e-5
  - 欠拟合 → 提高到 2e-4

学习率调度

```
LR_SCHEDULER="cosine_with_restarts"
LR_WARMUP_STEPS=40          # 预热步数 (总步数的 5%)
LR_SCHEDULER_NUM_CYCLES=1   # 重启次数
```

调度器选项:

- `cosine`: 余弦衰减（推荐）
- `cosine_with_restarts`: 余弦重启
- `constant`: 恒定学习率
- `linear`: 线性衰减

优化器

```
OPTIMIZER="AdamW8bit"        # 8bit AdamW 节省显存
```

选项:

- `AdamW8bit`: 节省显存（推荐）
- `AdamW`: 标准优化器
- `Lion`: 新型优化器
- `Prodigy`: 自适应学习率

精度设置

```
MIXED_PRECISION="fp16"       # 混合精度训练
SAVE_PRECISION="fp16"        # 模型保存精度
```

Bucket 设置（多分辨率）

```
--enable_bucket              # 启用 bucket 训练
--min_bucket_reso=256        # 最小分辨率
--max_bucket_reso=640        # 最大分辨率
--bucket_reso_steps=64       # 分辨率步长
```

其他重要参数

```
--cache_latents              # 缓存 VAE latents（加速训练）
--gradient_checkpointing     # 梯度检查点（节省显存）
--xformers                   # 启用 xformers 优化
--noise_offset=0.05          # 噪声偏移（提升细节）
--shuffle_caption            # 随机打乱标签顺序
--seed=42                    # 随机种子
```

训练结果

训练统计

- **总训练时间**: 约 50 分钟
- **总步数**: 820 步（3 epochs）
- **最终 Loss**: ~0.2
- **保存的模型**:
  - `nianha_lora.safetensors` (最终模型)
  - `nianha_lora-step00000273.safetensors` (Epoch 1)
  - `nianha_lora-step00000546.safetensors` (Epoch 2)

- `nianha_lora-step00000819.safetensors` (Epoch 3)

模型文件大小

```
nianha_lora.safetensors: ~50MB
```

Bucket 分布

训练时图片自动分配到不同分辨率 bucket：

```
Bucket 0: 320x576 - 20 张
Bucket 1: 384x512 - 1520 张 (主要)
Bucket 2: 448x448 - 10 张
Bucket 3: 512x384 - 20 张
Bucket 4: 576x320 - 50 张
```

推理相关

`inference_demo.py`

使用 diffusers 库的推理脚本。

特点:

- 现代化 API
- 支持 xformers
- 自动 LoRA 加载
- 批量生成

使用:

```
python inference_demo.py
```

可配置参数:

- LoRA 权重
- 推理步数
- CFG scale
- 图片尺寸
- 提示词列表

`inference_kohya.py`

使用 kohya-ss 工具的推理脚本。

特点:

- 调用 `gen_img_diffusers.py`
- 与训练工具兼容
- 命令行方式调用

使用:

```
python inference_kohya.py
```

数据集

`dataset/10_nianha/`

DreamBooth 格式的训练数据目录。

命名规则:

- `10_` 前缀表示重复 10 次
- `nianha` 是类别名称（触发词）

内容:

- 162 张 `.jpg` 图片
- 162 个对应的 `.txt` 标签文件

标签格式:

```
nianha, 年画局部, 仕女形象, 传统服饰, 细腻笔触
```

参考资源

官方文档

- kohya-ss/sd-scripts - 训练工具
- Diffusers 文档 - 推理库

训练指南

- LoRA 训练详解
- DreamBooth 训练指南

标签参考

- Danbooru Tags
- NovelAI Tag

社区

- CivitAI - LoRA 模型分享
- HuggingFace - 模型库